

METODE PENCARIAN *dan* **PELACAKAN**

Metode Pencarian

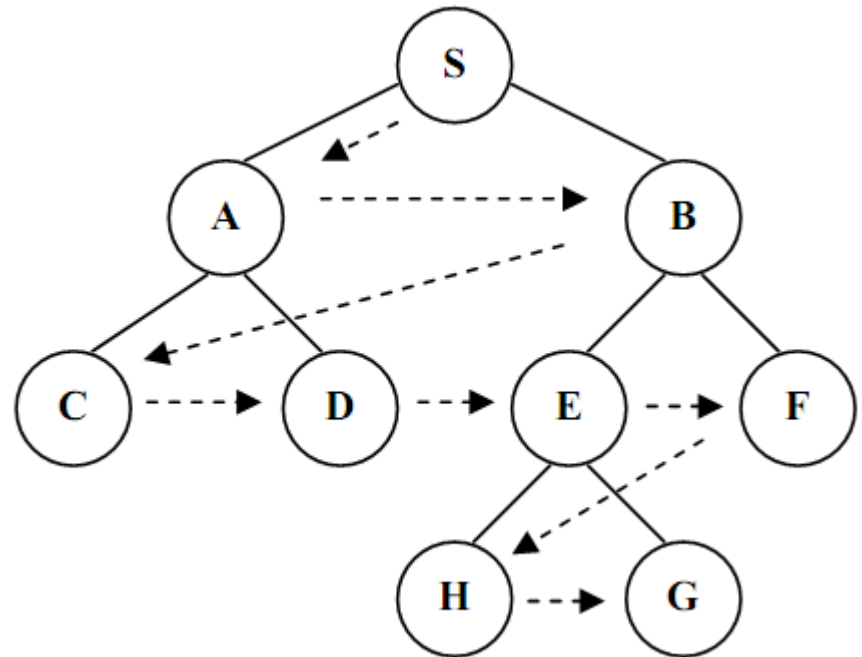
- Hal penting dalam menentukan keberhasilan sistem cerdas adalah kesuksesan dalam pencarian
- Pada dasarnya ada 2 Teknik pencarian :
 - A. Pencarian buta (blind search) :
 - 1. Pencarian melebar pertama (Breadth – First Search)
 - 2. Pencarian mendalam pertama (Depth – First Search)
 - B. Pencarian terbimbing (heuristic search) :
 - 1. Pendakian Bukit (Hill Climbing)
 - 2. Pencarian Terbaik Pertama (Best First Search)

A. Pencarian Buta (Blind Search)

1. Pencarian Melebar Pertama
(Breadth – First Search)
 - ⦿ Semua node pada level n akan dikunjungi terlebih dahulu sebelum mengunjungi node-node pada level $n+1$. pencarian dimulai dari node akar terus ke level 1 dari kiri ke kanan, kemudian berpindah ke level berikutnya dari kiri ke kanan hingga solusi ditemukan.

Lanjutan...

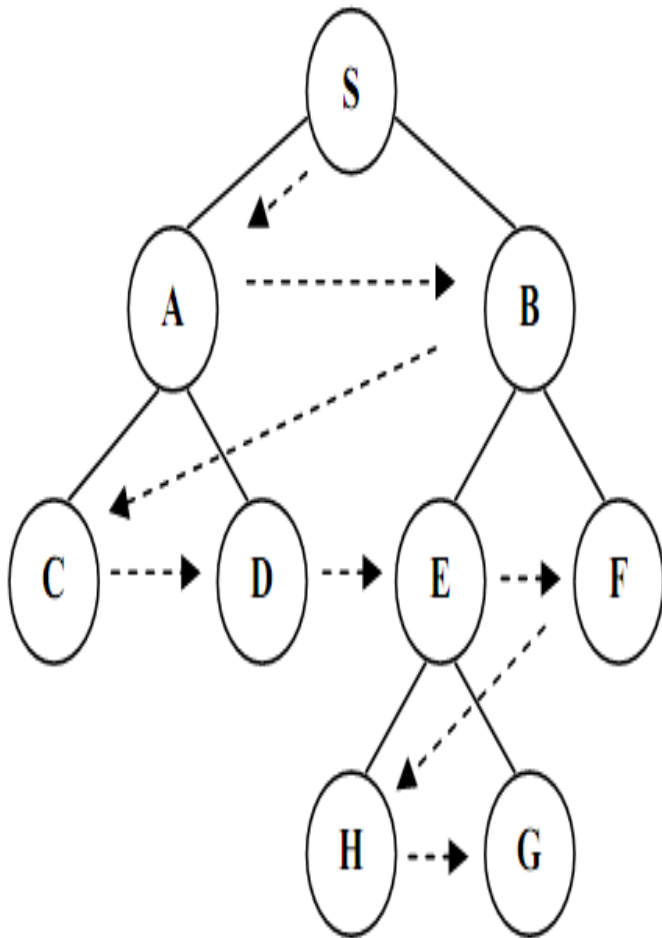
Gambar disamping merupakan gambar pencarian dengan BFS



Lanjutan...

Algoritma :

1. Buat suatu variabel **Node_List** dan tetapkan sebagai keadaan awal
2. Kerjakan langkah-langkah berikut ini sampai tujuan tercapai atau **node_list** dalam keadaan kosong:
 - a. Tetapkan elemen pertama dari **Node_List** tersebut dengan nama **S** Jika **Node_List** kosong, keluar.
 - b. Pada setiap langkah yang aturannya cocok dengan **S**, kerjakan
 - i. Aplikasikan aturan tersebut untuk membentuk suatu keadaan baru
 - ii Jika keadaan awal adalah tujuan yang diharapkan, sukses dan keluar
 - iii Jika Tidak, tambahkan keadaan awal yang baru tersebut pada akhir **Node_List**.



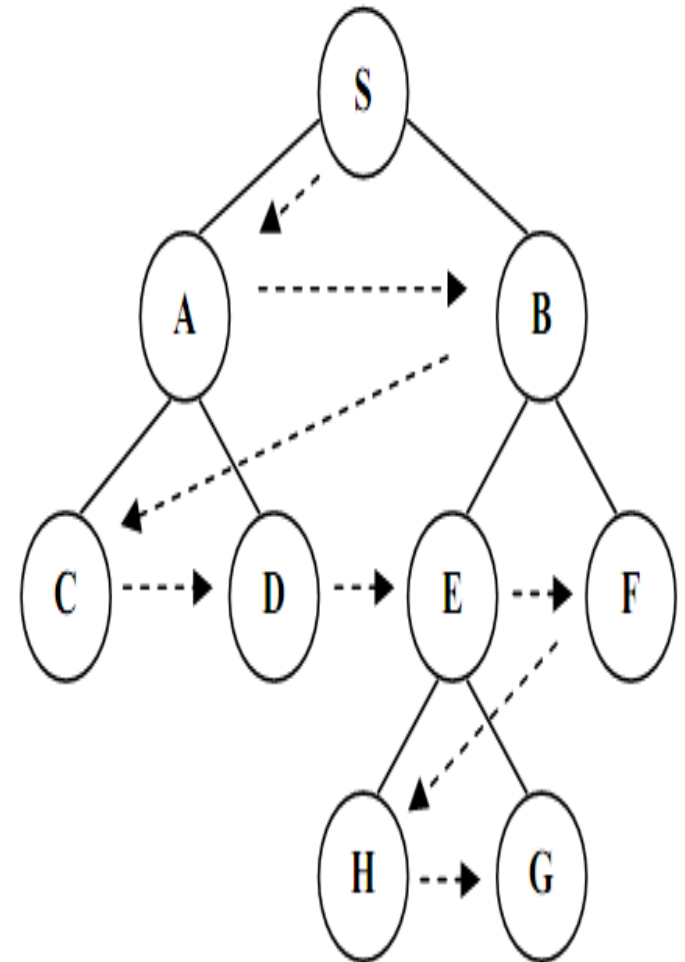
lanjutan...

Keuntungan :

- Tidak akan menemui jalan buntu
- Jika ada 1 solusi, maka BFS akan menemukannya, Dan jika ada lebih dari 1 solusi, maka solusi minimum akan ditemukan.

Kelemahan :

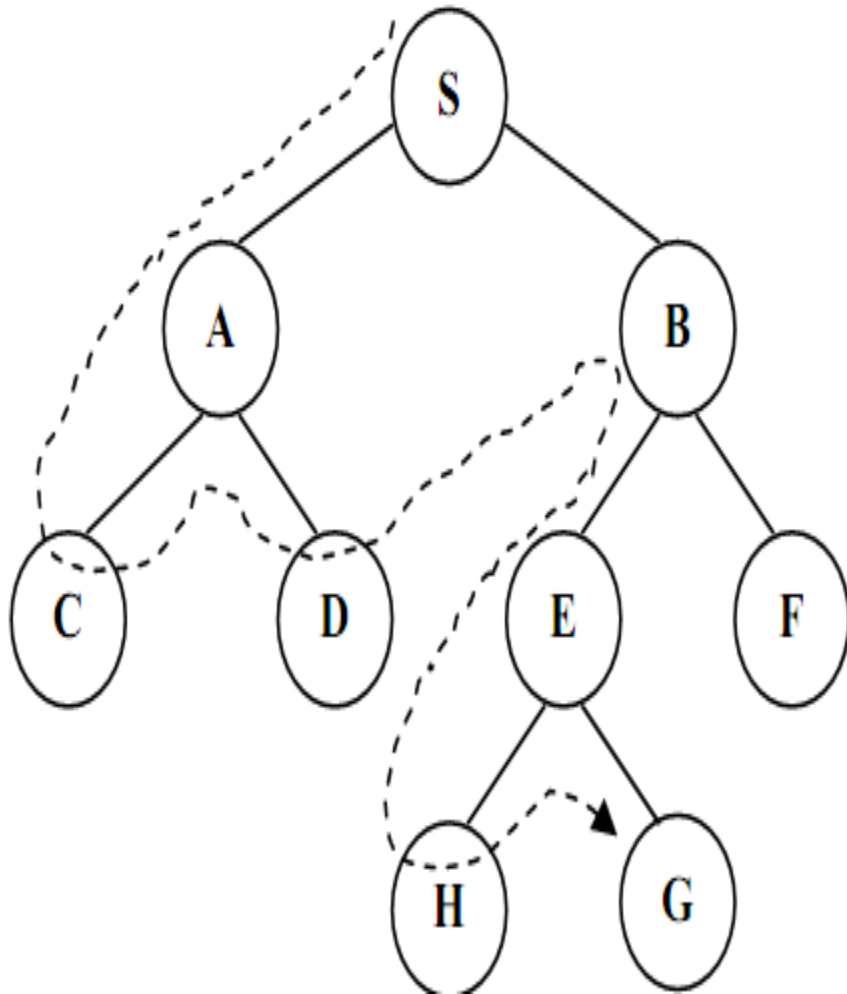
- Membutuhkan memori yang cukup banyak karena menyimpan semua node dalam satu pohon
- Membutuhkan waktu yang cukup lama, karena akan menguji n level untuk mendapatkan solusi pada level yang ke $(n+1)$



2. Pencarian Mendalam Pertama (Depth-First Search/DFS)

Proses pencarian dilakukan pada semua anaknya sebelum dilakukan pencarian ke node-node yang selevel. Pencarian dimulai dari node akar ke level yang lebih tinggi. Proses diulangi terus hingga ditemukan solusi.

Lanjutan...

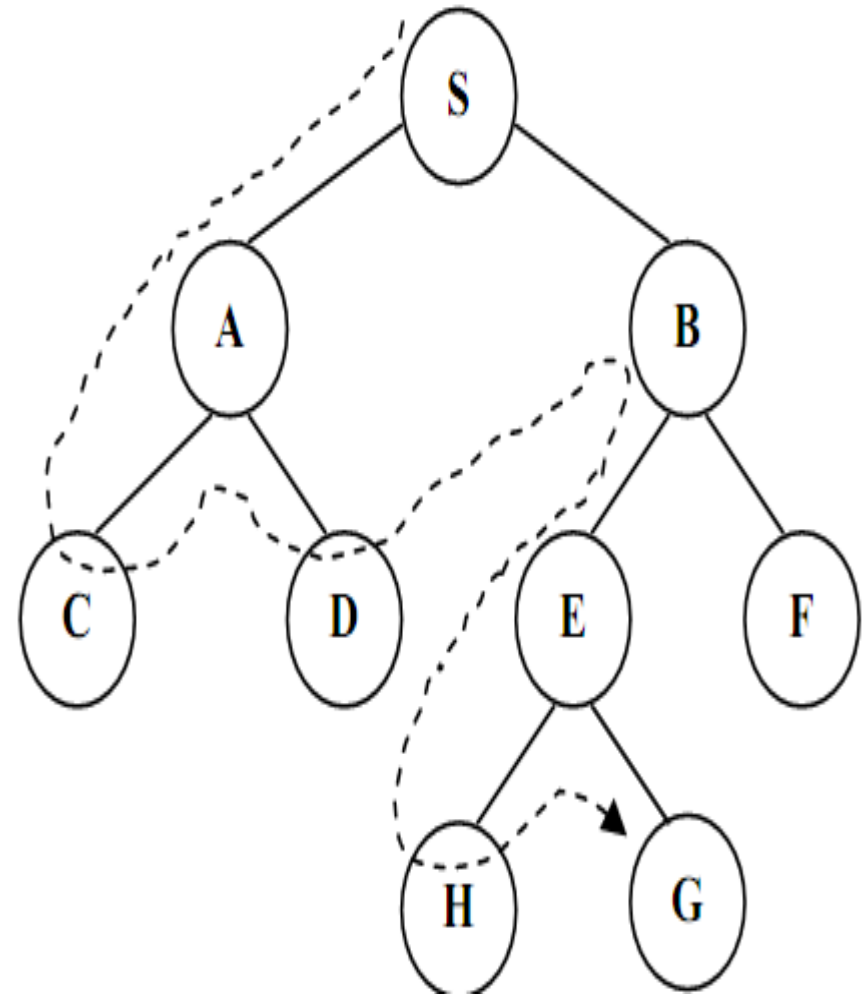


- Keterangan di atas dapat diperlihatkan pada gambar disamping ini

Lanjutan...

◉ Algoritma :

1. Jika keadaan awal merupakan tujuan, keluar (sukses)
2. Jika tidak, kerjakan langkah-langkah berikut ini sampai tercapai keadaan sukses atau gagal :
 - a. Bangkitkan succesor S dari keadaan awal. Jika tidak ada succesor, maka akan terjadi kegagalan.
 - b. Panggil DFS dengan S sebagai keadaan awal
 - c. Jika sukses berikan tanda sukses. Namun jika tidak, ulangi langkah 2



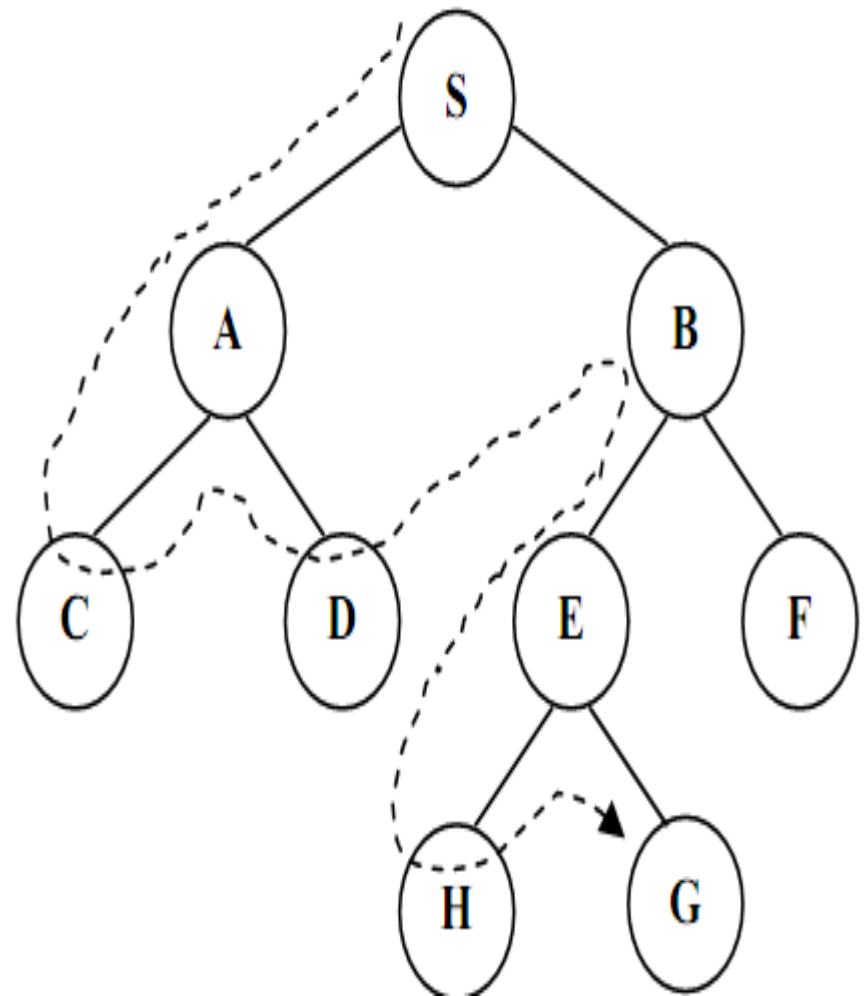
Lanjutan...

Keuntungan :

- Membutuhkan memori relatif kecil, karena hanya node-node pada lintasan yang aktif saja yang disimpan
- Secara kebetulan, akan menemukan solusi tanpa harus menguji lebih banyak lagi dalam ruang keadaan

Kelemahan :

- Memungkinkan tidak ditemukannya tujuan yang diharapkan
- Hanya mendapat 1 solusi pada setiap pencarian



B. Pencarian Heuristik (Heuristic Search)

Pencarian buta tidak selalu dapat diterapkan dengan baik, hal ini disebabkan waktu aksesnya yang cukup lama & besarnya memori yang diperlukan. Kelemahan ini dapat diatasi jika ada informasi tambahan dari domain yang bersangkutan.

Lanjutan...

Misal kasus 8-puzzle. Ada 4 operator yang dapat digunakan untuk menggerakkan dari satu keadaan ke keadaan yang baru

1. Ubin kosong digeser ke kiri
2. Ubin kosong digeser ke kanan
3. Ubin kosong digeser ke atas
4. Ubin kosong digeser ke bawah

1	2	3
7	8	4
6		5

Lanjutan...

Keadaan awal

1	2	3
7	8	4
6		5



Tujuan

1	2	3
8		4
7	6	5

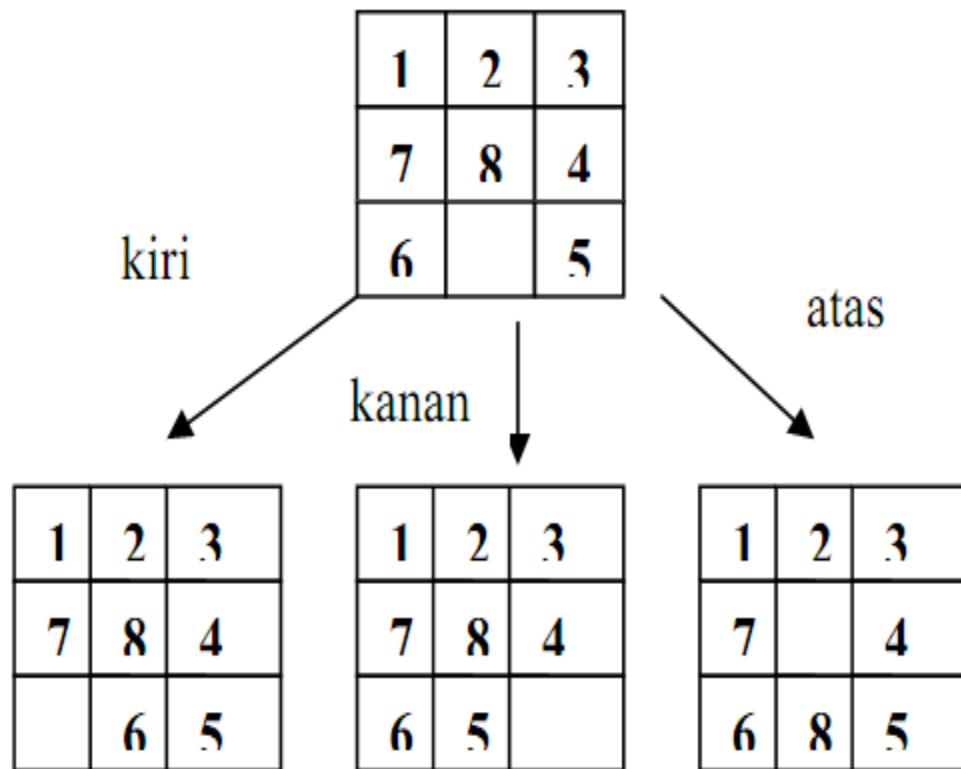
Kasus 8 puzzle

Lanjutan...

Langkah awal

Tujuan

1	2	3
8		4
7	6	5



Langkah awal kasus 8 puzzle

Lanjutan...

Pada langkah awal dari permainan puzzle di atas hanya ada 3 operator yang bisa digunakan, yaitu ubin kosong digeser ke kiri, ke kanan dan ke atas. Apabila digunakan pencarian buta, kita tidak perlu mengetahui operasi apa yang sedang dikerjakan (sembarang operasi bisa digunakan)

Pada pencarian heuristik perlu diberikan informasi khusus, yaitu :

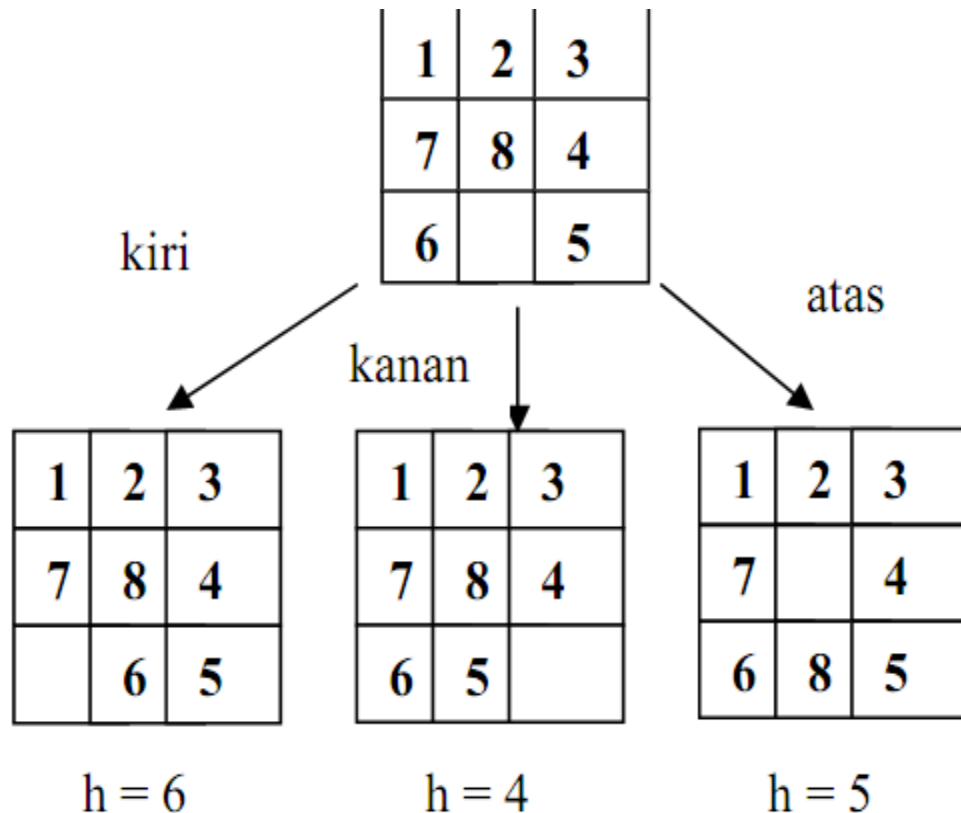
- ◆ Untuk jumlah ubin yang menempati posisi yang benar Jumlah yang lebih tinggi adalah yang lebih diharapkan (lebih baik)

Lanjutan...

- Untuk jumlah ubin yang menempati posisi yang benar
Jumlah yang lebih tinggi adalah yang lebih diharapkan
(lebih baik)

Tujuan

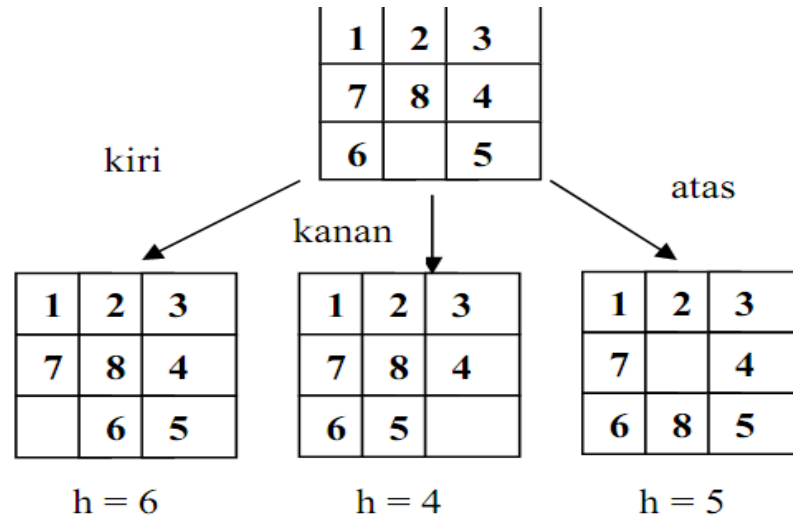
1	2	3
8		4
7	6	5



Lanjutan...

Tujuan

1	2	3
8		4
7	6	5



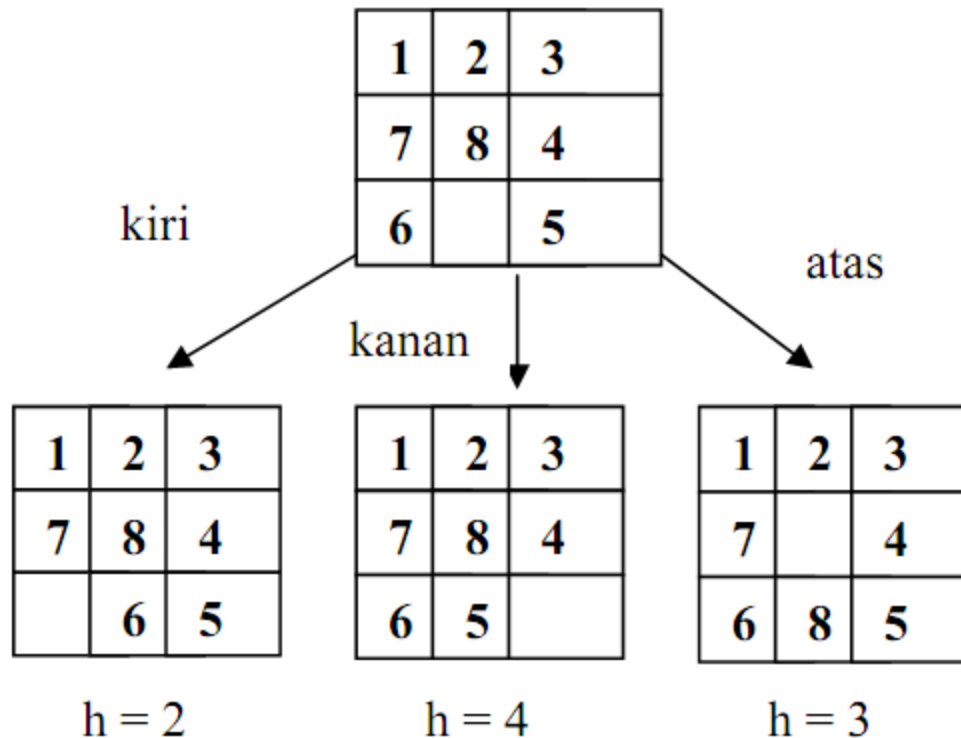
Dari 3 operator yang digunakan diperoleh hasil pergeseran ubin kosong ke kiri bernilai 6 ($h=6$), pergeseran kosong ke kanan bernilai 4 ($h=4$) dan pergeseran ubin kosong ke atas bernilai 5 ($h=5$), sehingga langkah selanjutnya yang harus dilakukan adalah menggeser ubin kosong ke kiri

Lanjutan...

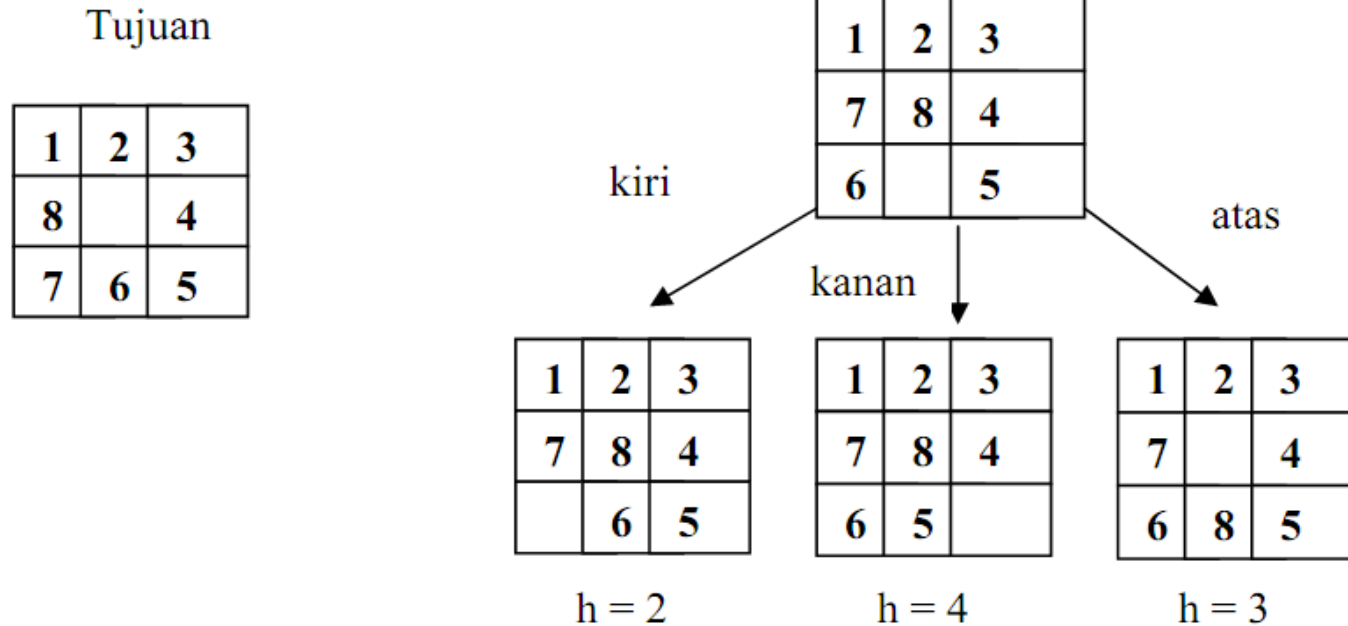
- ♦ Untuk jumlah ubin yang menempati posisi yang salah
Jumlah yang lebih kecil adalah yang diharapkan (lebih baik)

Tujuan

1	2	3
8		4
7	6	5



Lanjutan...



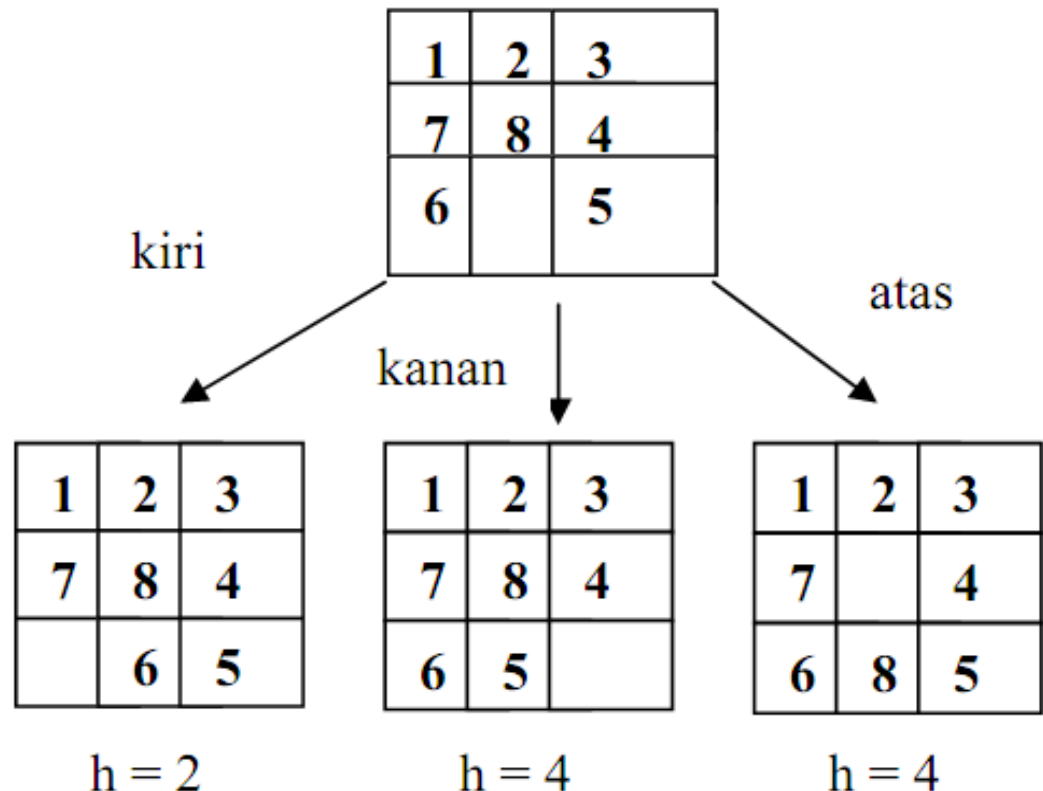
Nilai terbaik diperoleh dengan menggeser ubin kosong ke kiri ($h=2$) karena nilai ini yang paling kecil diantara nilai lainnya (4 dan 3), sehingga langkah selanjutnya adalah menggeser ubin kosong ke kiri

Lanjutan...

- ♦ Menghitung total gerakan yang diperlukan untuk mencapai tujuan. Jumlah yang lebih kecil adalah yang diharapkan (lebih baik)

Tujuan

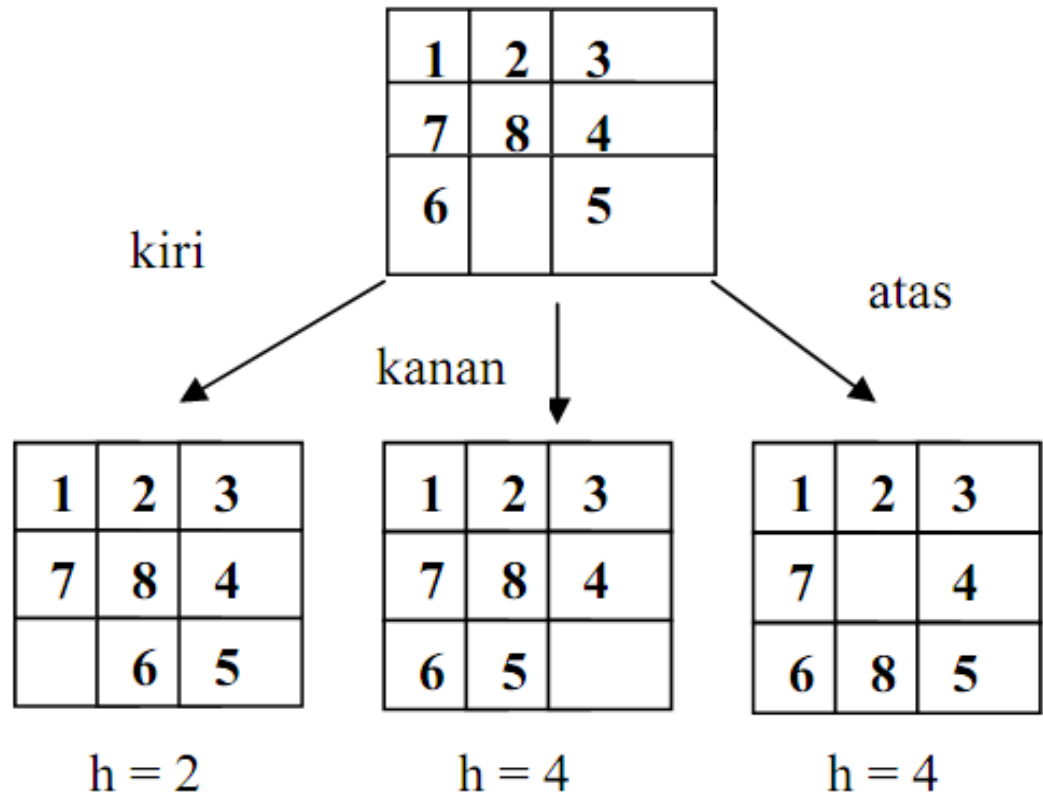
1	2	3
8		4
7	6	5



Lanjutan...

Tujuan

1	2	3
8		4
7	6	5



- ◆ Nilai terbaik adalah 2 ($h=2$) yang diperoleh dengan menggeser ubin kosong ke kiri, nilai ini paling kecil dibanding dengan nilai lainnya(4), sehingga menggeser ubin kosong ke kiri adalah langkah yang harus dilakukan selanjutnya

Lanjutan...

Ada 4 metode pencarian Heuristik

1. Pembangkitan dan Pengujian (*Generate and Test*)
2. **Pendakian Bukit (*Hill Climbing*)**
3. **Pencarian Terbaik Pertama (*Best First Search*)**
4. *Simulated Annealing*

B1. Pendakian Bukit (*Hill Climbing*)

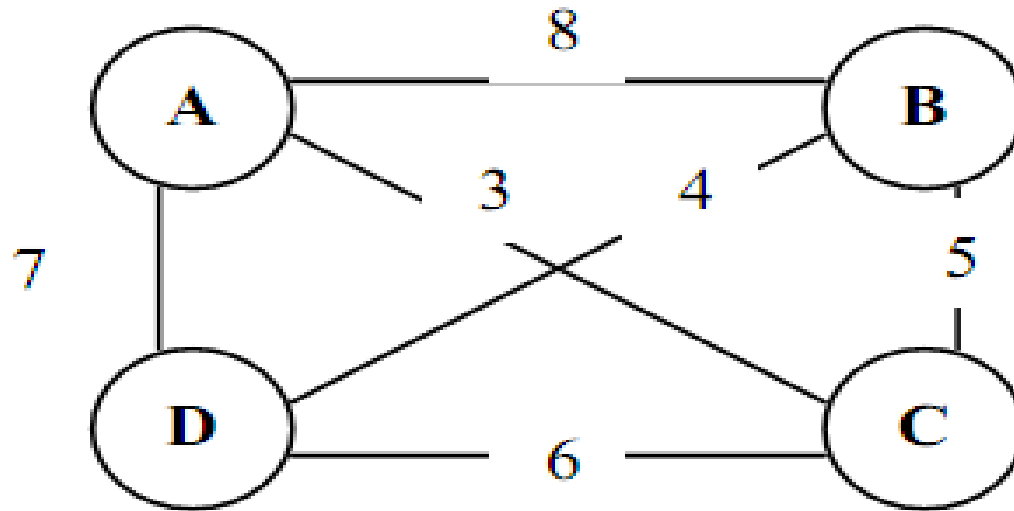
Contoh : Traveling Salesman Problem (TSP)

Seorang salesman ingin mengunjungi n kota. Jarak antara tiap-tiap kota sudah diketahui. Kita ingin mengetahui rute terpendek dimana setiap kota hanya boleh dikunjungi tepat 1 kali.

Lanjutan...

Misal :

ada 4 kota dengan jarak antara tiap-tiap kota seperti berikut ini :



Lanjutan...

Solusi – solusi yang mungkin dengan menyusun kota-kota dalam urutan abjad, misal :

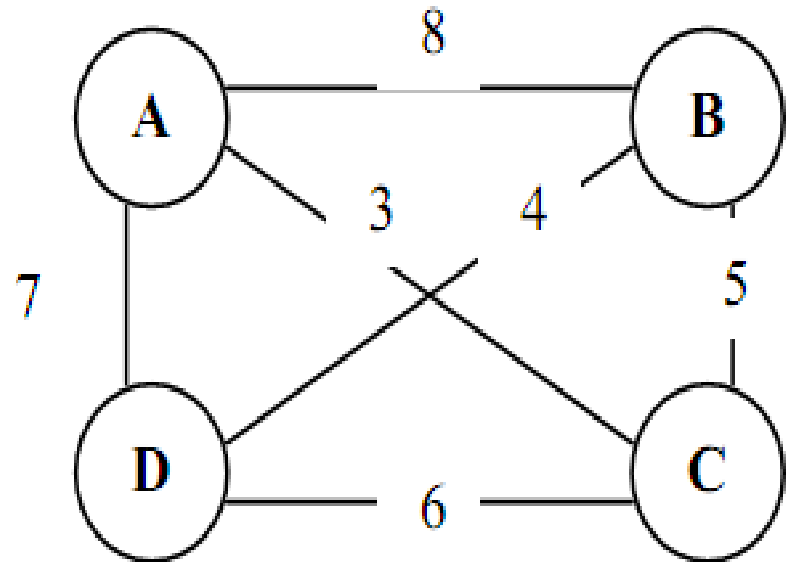
A – B – C – D : dengan panjang lintasan (=19)

A – B – D – C : (=18)

A – C – B – D : (=12)

A – C – D – B : (=13)

dst



a. *Metode Simple Hill Climbing*

Ruang keadaan berisi semua kemungkinan lintasan yang mungkin. Operator digunakan untuk menukar posisi kota-kota yang bersebelahan.

Fungsi heuristik yang digunakan adalah panjang lintasan yang terjadi.

Operator yang akan digunakan adalah menukar urutan posisi 2 kota dalam 1 lintasan. Bila ada n kota, dan ingin mencari kombinasi lintasan dengan menukar posisi urutan 2 kota, maka akan didapat sebanyak :

$$\frac{n!}{2!(n-2)!} = \frac{4!}{2!(4-2)!} = 6 \text{ kombinasi}$$

Lanjutan...

Keenam kombinasi ini akan dipakai semuanya sebagai operator, yaitu :

Tukar 1,2 = menukar urutan posisi kota ke – 1 dengan kota ke – 2

Tukar 2,3 = menukar urutan posisi kota ke – 2 dengan kota ke – 3

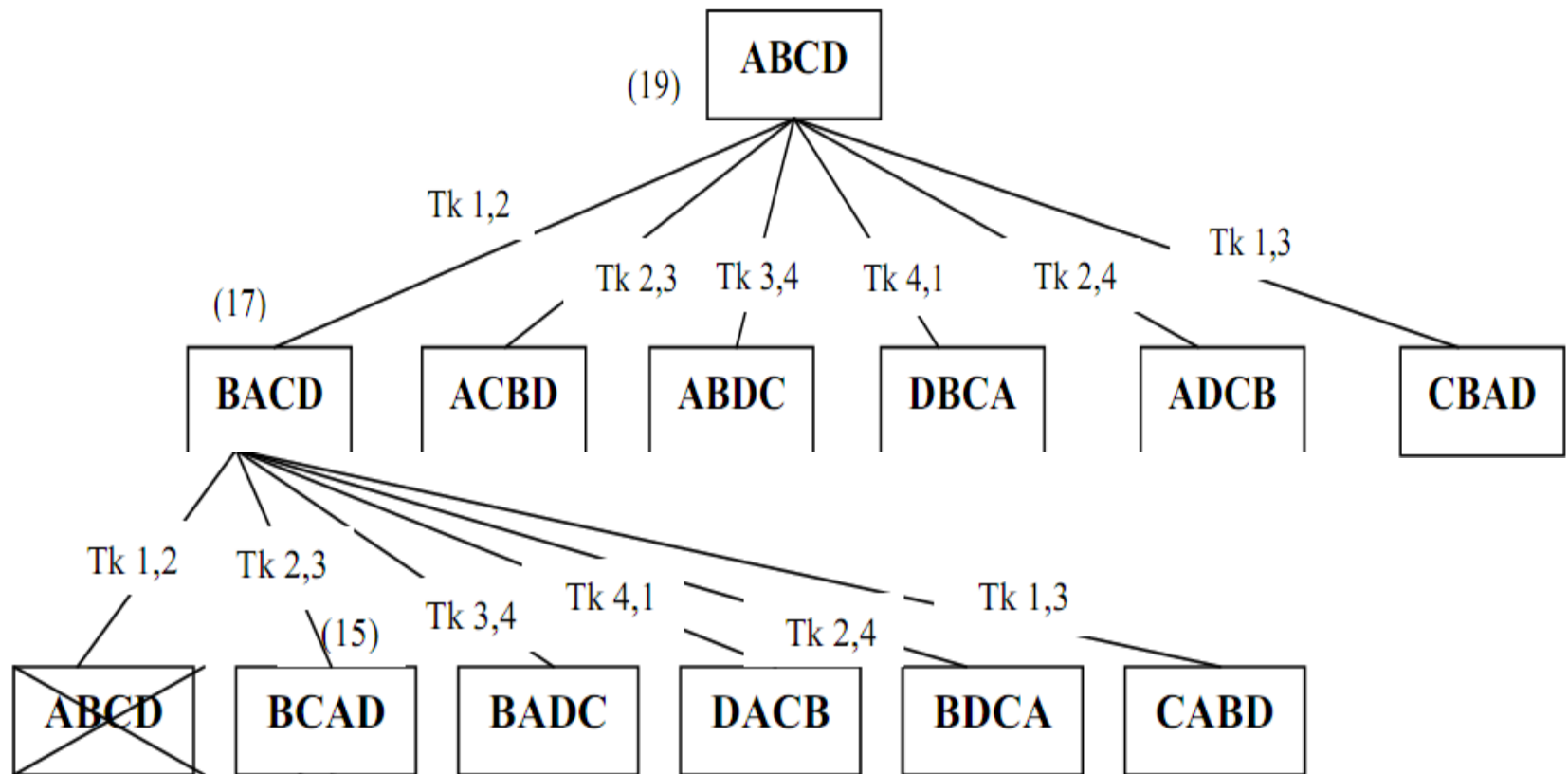
Tukar 3,4 = menukar urutan posisi kota ke – 3 dengan kota ke – 4

Tukar 4,1 = menukar urutan posisi kota ke – 4 dengan kota ke – 1

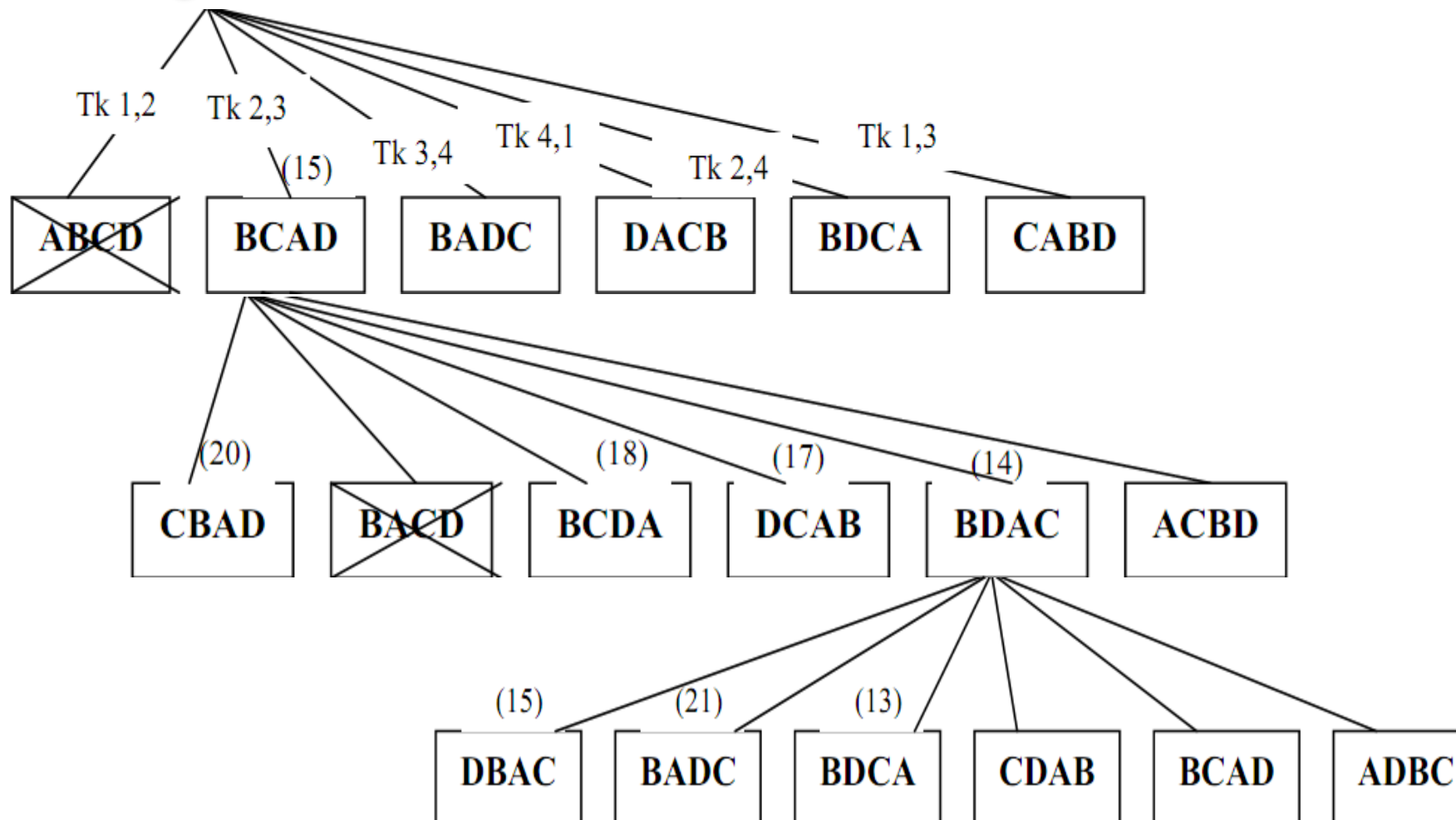
Tukar 2,4 = menukar urutan posisi kota ke – 2 dengan kota ke – 4

Tukar 1,3 = menukar urutan posisi kota ke – 1 dengan kota ke – 3

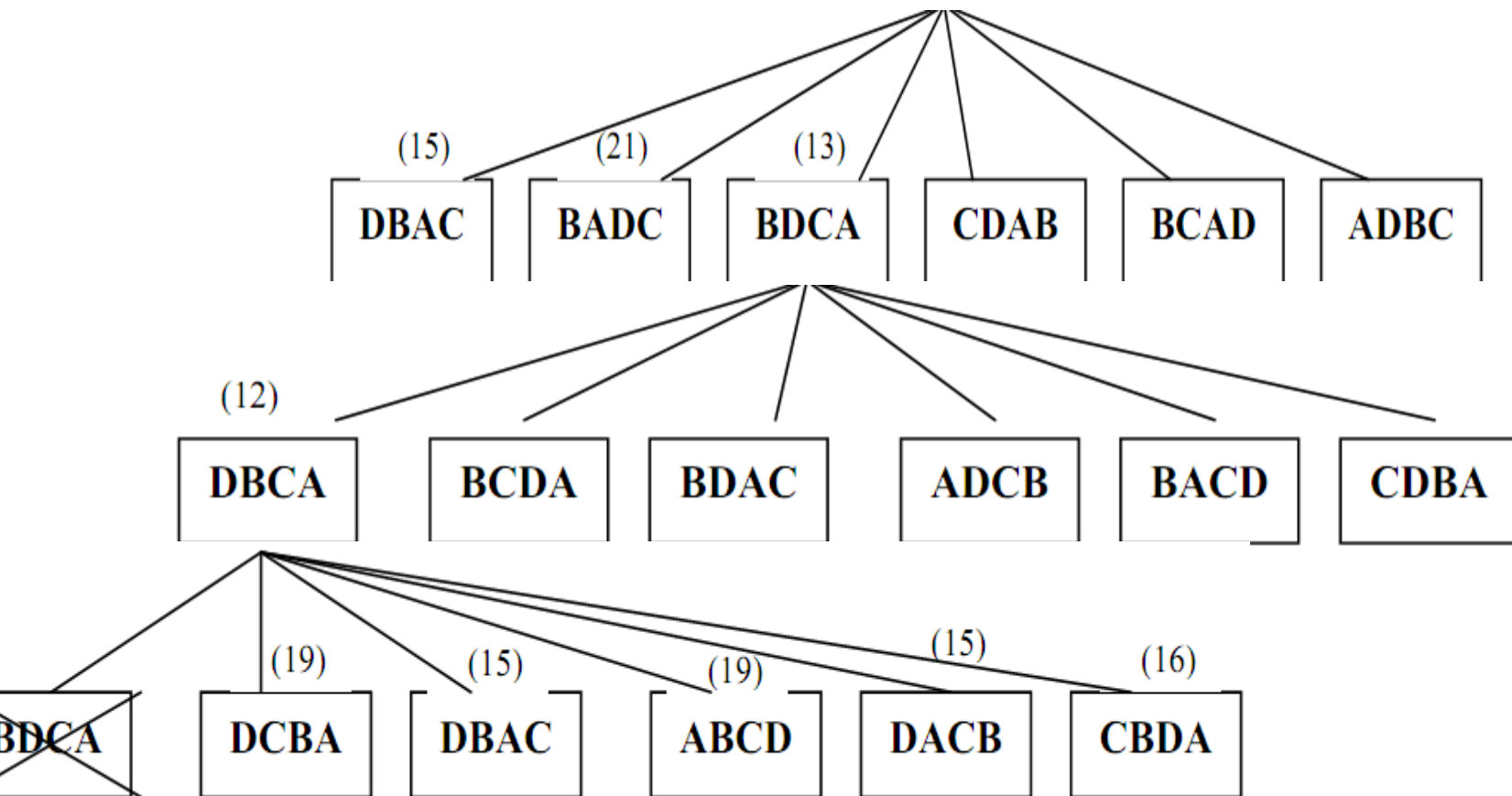
Lanjutan...



Lanjutan...



Lanjutan...



Lanjutan...

- Keadaan awal, lintasan ABCD (=19).
Level pertama, hill climbing mengunjungi BACD (=17), BACD (=17) < ABCD (=19), sehingga BACD menjadi pilihan selanjutnya dengan operator Tukar 1,2
- Level kedua, mengunjungi ABCD, karena operator Tukar 1,2 sudah dipakai BACD, maka pilih node lain yaitu BCAD (=15), BCAD (=15) < BACD (=17)
- Level ketiga, mengunjungi CBAD (=20), CBAD (=20) > BCAD (=15), maka pilih node lain yaitu BCDA (=18), pilih node lain yaitu DCAB (=17), pilih node lain yaitu BDAC (=14),
BDAC (=14) < BCAD (=15)

Lanjutan...

- Level keempat, mengunjungi DBAC (=15), DBAC(=15) > BDAC (=14), maka pilih node lain yaitu BADC (=21), pilih node lain yaitu BDCA (=13), BDCA (=13) < BDAC (=14)
- Level kelima, mengunjungi DBCA (=12), DBCA (=12) < BDCA (=13)
- Level keenam, mengunjungi BDCA, karena operator Tukar 1,2 sudah dipakai DBCA, maka pilih node lain yaitu DCBA, pilih DBAC, pilih ABCD, pilih DACB, pilih CBDA

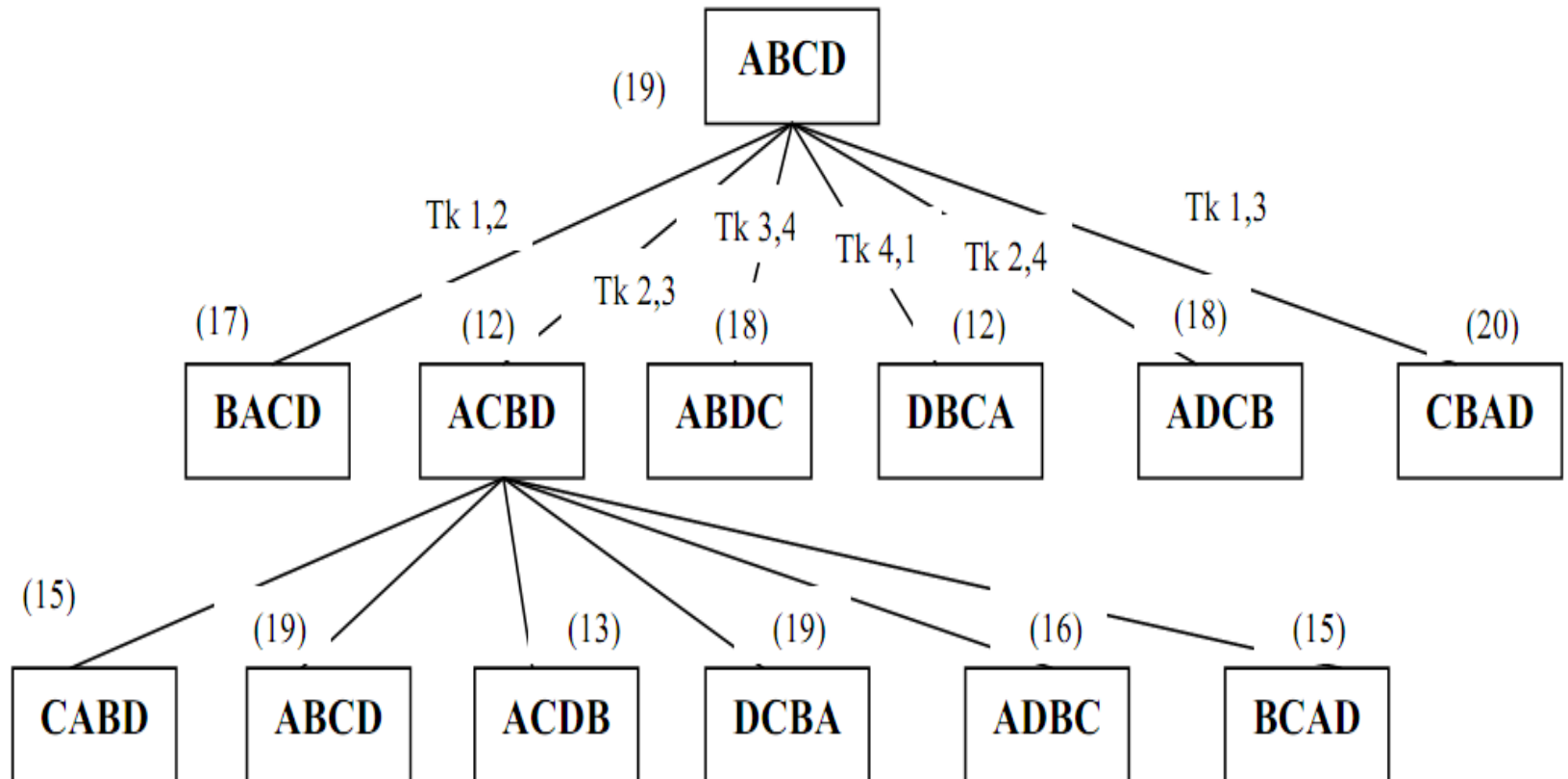
Lanjutan...

Karena sudah tidak ada node yang memiliki nilai heuristik yang lebih kecil dibanding nilai heuristik DBCA, maka node DBCA (=12) adalah lintasan terpendek yang merupakan (SOLUSI)

b. Metode steepest hill climbing

- Steeppest hill climbing hampir sama dengan simple hill climbing, hanya saja gerakan pencarian tidak dimulai dari kiri, tetapi berdasarkan nilai heuristik terbaik.
- Perhatikan gambar berikut ini !

Lanjutan...



Lanjutan...

- ◉ Keadaan awal, lintasan ABCD (=19).
- ◉ Level pertama, hill climbing memilih nilai heuristik terbaik yaitu ACBD (=12) sehingga ACBD menjadi pilihan selanjutnya.
- ◉ Level kedua, hill climbing memilih nilai heuristik terbaik, karena nilai heuristik lebih besar dibanding
- ◉ ACBD, maka hasil yang diperoleh lintasannya tetap ACBD (=12) sebagai solusi

B2. Best First Search

- ⦿ Metode best first search merupakan kombinasi dari metode depth first search dan breadth first search
- ⦿ Dengan mengambil kelebihan dari kedua metode tersebut. Hill climbing tidak diperbolehkan untuk kembali ke node pada level lebih rendah meskipun node tersebut memiliki nilai heuristik lebih baik.

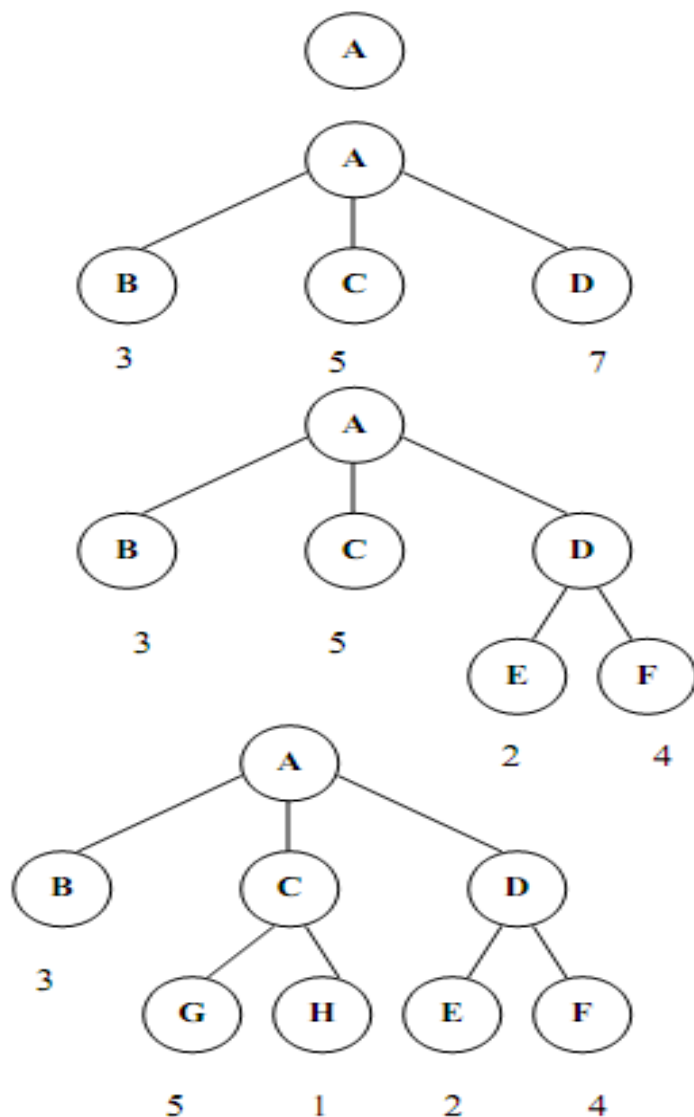
Lanjutan...

- ⦿ Pada best first search, pencarian diperbolehkan mengunjungi node di level lebih rendah, jika ternyata node di level lebih tinggi memiliki nilai heuristik lebih buruk.
- ⦿ Untuk mengimplementasikan metode ini, dibutuhkan 2 antrian yang berisi node-node, yaitu :

Lanjutan...

- ◉ OPEN : berisi node-node yang sudah dibangkitkan, sudah memiliki fungsi heuristik namun belum diuji. Umumnya berupa antrian berprioritas yang berisi elemen-elemen dengan nilai heuristik tertinggi.
- ◉ CLOSED : berisi node-node yang sudah diuji.

Lanjutan...



Antrian OPEN

[A]

[D C B]

[C F B E]

[G F B E H]

Diasumsikan node dengan nilai yang lebih besar memiliki nilai evaluasi yang lebih baik. Pada keadaan awal, antrian berisi A. Pengujian dilakukan di level pertama, node D memiliki nilai terbaik, sehingga menempati antrian pertama, disusul dengan C dan B. Node D memiliki cabang E dan F yang masing-masing bernilai 2 & 4. Dengan demikian C merupakan pilihan terbaik dengan menempati antrian pertama. Demikian seterusnya.

A full-page background image showing a beach at sunset or sunrise. The sky is a gradient of orange and yellow. Two large, dark rock formations stand in the water, their silhouettes reflected in the calm, wet sand. The overall color palette is dominated by warm, golden tones.

Terima Kasih